Distance-Preserving Digests: A Primitive for BFT Consensus*

Ryan Mercier

School of Engineering, University of Connecticut, Storrs, CT, USA
ryan.mercier@uconn.edu

Abstract. Every BFT consensus protocol uses collision-resistant hashes to compare validator state. Collision resistance destroys distance: two validators agreeing on 19 of 20 transactions produce unrelated hashes, indistinguishable from validators sharing nothing. This forces three design constraints across the BFT literature: validators must synchronize state before voting, agreement quality cannot be measured until votes are counted, and hierarchical committees must be large enough for independent BFT, limiting tree depth. This paper introduces distance-preserving transaction digests, a primitive that replaces the collision-resistant hash with a commutative vector sum: each transaction hashes to a point in an 8-dimensional space, and a validator's digest is the sum of the points for the transactions it holds, so the Euclidean distance between two digests is proportional to the number of transactions on which the two validators differ. Beyond proportional distance, the primitive has two further properties hashes lack: weighted means of digests summarize a group exactly, and set differences are identifiable via bloom filter diff. We demonstrate three applications: a two-phase BFT protocol (Proxima) that achieves single-round finality when validators agree; tree-structured consensus with groups of 10 validators (vs 128 in Ethereum), enabled because distance filtering replaces per-group BFT; and cross-shard consistency verification at 128 bytes per shard pair, replacing the per-transaction coordination of two-phase commit. Safety is proved: fewer than $N/3$ Byzantine validators cannot cause conflicting finalization, independent of Phase 1 clustering or tree topology. At N=100,000, Proxima Tree uses 2.2× fewer messages than HotStuff (a structural property unaffected by parallelism). Single-core finality is ∼0.9s vs ∼18s for HotStuff; multi-core BLS narrows but does not eliminate this gap.

Keywords: Byzantine fault tolerance · blockchain · consensus protocols · locality-sensitive hashing · sharding

1 Introduction

BFT consensus has been built on collision-resistant hashing since PBFT [5]. Collision resistance is the right property for preventing forgery, but the wrong

* Code and reproduction artifacts: https://github.com/RyanMercier/Proxima

property for measuring agreement. When two validators compute SHA-256 of their transaction sets and the hashes differ, the protocol learns exactly one bit of information: not identical. It cannot tell whether they differ on one transaction or half the transactions. It cannot measure how close they are to agreement. It cannot summarize a group of validators as a single compact value.

These limitations are baked into every protocol that uses hashes for state comparison. Consider HotStuff [25], the leader-based BFT protocol underlying several production systems: leadership rotates among the validators, and in each view the leader drives the block through three voting rounds, collecting the votes of each round into a threshold-signature quorum certificate that proves $2N/3$ support. Because those certificates are built from collision-resistant hashes, HotStuff runs all three rounds even when every validator agrees perfectly; unanimous agreement is undetectable until all votes are counted. Ethereum 2.0 [12] requires committees of 128 validators because each committee must independently reach 2/3 consensus, and small committees fail when random assignment concentrates Byzantine validators. Cross-shard transactions require multi-round coordination (two-phase commit or receipt chains) because hashes cannot verify partial overlap between shard states.

This paper proposes replacing collision-resistant hashes with distance-preserving digests for state comparison in BFT protocols. The primitive is simple: hash each transaction with SHA-512, split the 64-byte output into 8 segments, treat each as a coordinate, and sum across transactions. The result is a commutative vector in 8D space where Euclidean distance is proportional to transaction disagreement. The primitive is not novel in the data-structures sense (locality-sensitive hashing dates to Indyk and Motwani [16]), but its application to BFT consensus is, to our knowledge, new.

We demonstrate that this single primitive removes all three constraints. Agreement quality is measurable in one round (enabling fast-path finality). Hierarchical groups need not reach internal BFT (enabling groups of 10 and deeper trees). Cross-shard consistency is verifiable at constant cost per shard pair. We implement these as Proxima, a working BFT blockchain, and evaluate message complexity, bandwidth, and projected latency against HotStuff, PBFT, and Ethereum's committee structure.

Throughout, we analyze Proxima under a static adversary that chooses which validators to corrupt before the protocol begins. Static corruption is the standard first-analysis setting for this class of protocols: it is the model in which the classical safety arguments for PBFT, HotStuff, and Tendermint are stated, and it isolates what the new primitive contributes from the orthogonal problem of corruption that reacts to protocol state. Adaptive adversaries, which corrupt validators after observing information such as tree assignments, are out of scope for this paper; Sect. 11 discusses what defending against them requires.

2 Related Work

BFT Consensus. PBFT [5] established practical BFT with $O(N^2)$ message complexity. HotStuff [25] reduced this to $O(N)$ using leader-based aggregation and BLS threshold signatures. Tendermint [4] provides a similar $O(N)$ BFT protocol. All use collision-resistant hashes for state comparison and require full state synchronization before voting. Proxima shares HotStuff's $O(N)$ voting complexity but adds Phase 1 distance-based pre-filtering and bloom-based synchronization. Proxima's fast path follows the speculative fast-path lineage of Zyzzyva [17] and SBFT [15], in which signed commitments collected optimistically finalize a request in fewer rounds whenever a large enough quorum already agrees.

Hierarchical BFT. Kauri [19] applies tree aggregation to HotStuff, reducing the leader bottleneck. Kauri's tree nodes still run per-group BFT (hash-based), requiring groups large enough for Byzantine tolerance. Proxima's distance filtering removes this requirement, enabling groups of 10 vs 128 and deeper trees (5 levels vs 2–3). Ethereum 2.0 [12] uses random committees of 128 with per-committee BFT attestation; Proxima achieves similar distributed aggregation with groups 12.8× smaller.

DAG-Based Protocols. Narwhal and Tusk [7] decouple data availability from consensus via a DAG of validator proposals. Bullshark [22] builds on Narwhal with a simpler consensus rule. These protocols solve a different problem but share the property that validators may have partial views. Digests could augment DAG protocols by measuring vertex agreement quality.

Locality-Sensitive Hashing. LSH was introduced by Indyk and Motwani [16] for approximate nearest-neighbor search. SimHash [6] applies it to document similarity; MinHash [3] to Jaccard similarity; p-stable LSH [8] to L_p distances. Our digest is a simple LSH scheme applied to transaction sets. No work in the LSH literature applies it to Byzantine agreement, and no work in the consensus literature reaches for distance-preserving functions.

Sharding. Ethereum abandoned execution sharding in favor of data-availability sharding (Danksharding [11]). NEAR's Nightshade [21] uses receipt-based communication. Shardeum uses an address-range approach with atomic cross-shard composability. Digest-based verification offers a complementary approach: constant-cost verification per shard pair with conflict resolution only for divergent transactions.

3 System Model

We consider a system of N validators, of which at most $f < N/3$ are Byzantine. Byzantine validators may behave arbitrarily. Honest validators follow the protocol.

Network model. We assume partial synchrony [10]: there exists an unknown Global Stabilization Time (GST) after which all messages between honest validators are delivered within a known bound Δ. This is the standard model for PBFT [5], HotStuff [25], and Tendermint [4].

Cryptographic assumptions. We assume a BLS signature scheme [2] where each validator holds a private key and the corresponding public key is known to all. BLS signatures support aggregation: any party can combine N individual signatures into a single 96-byte aggregate signature. We assume the hardness of the CDH problem in the BLS12-381 pairing group [23].

Communication model. Validators communicate through an aggregator (flat mode) or through a tree of relay nodes (tree mode). The aggregator/relay role rotates each round. A Byzantine aggregator can suppress messages (affecting liveness) but cannot forge BLS signatures (safety is maintained).

Adversary model. The adversary is static: it chooses which validators to corrupt before the protocol begins. The adversary has full network visibility and can coordinate Byzantine strategies.

Hash function model. We model SHA-512 as producing outputs uniformly distributed over $\{0,1\}^{512}$. Each 64-bit segment, when reduced modulo 10000 and divided by 10000, produces a value uniformly distributed in $[0,1)$.

Partial observation parameter. Honest validators may receive transactions with delay relative to the proposer. We parameterize this by p_{miss}, the per-validator probability of having an incomplete view at proposal time. We use $p_{\text{miss}} = 0.37$ throughout, drawn from internal measurements; this is conservative relative to typical mainnet conditions.

4 The Primitive: Distance-Preserving Digests

The digest must do two jobs at once: stay as cheap to compute and transmit as an ordinary hash, and turn set disagreement into measurable geometry. We first give the construction, then the three properties that the rest of the paper builds on, and finally the calibration procedure and liveness analysis that make a distance threshold usable when some of the parties choosing their digests are adversarial.

4.1 Construction

Each transaction tx is hashed with SHA-512, producing 64 bytes. The output is split into 8 segments of 8 bytes each. Each segment is interpreted as an unsigned integer, reduced modulo 10000, and divided by 10000 to produce a coordinate in $[0,1)$:

$$v(tx) = \left[\, \text{seg}_0/10000,\ \ldots,\ \text{seg}_7/10000 \,\right]$$

A validator's digest for a set of transactions T is the commutative sum $D(T) = \sum_{tx \in T} v(tx)$.

4.2 Properties

Property 1: Proportional distance. If validators A and B have transaction sets differing by k transactions, the Euclidean distance between $D(T_A)$ and

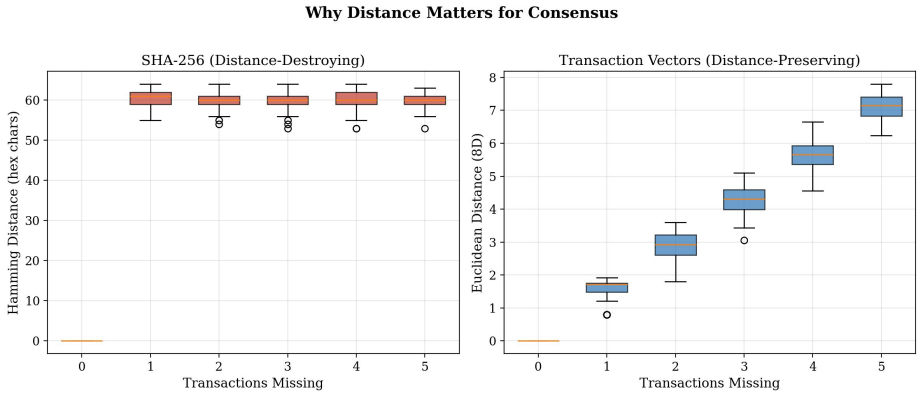

Fig. 1. SHA-256 is distance-destroying; dropping even one transaction produces a completely unrelated hash. Transaction vectors are distance-preserving; distance grows proportionally with the number of missing transactions.

$D(T_B)$ grows proportionally with k. Empirically, $k = 1$ gives ~ 1.6, $k = 2$ gives ~ 3.0, $k = 10$ gives ~ 14.5.

Property 2: Exact summarization. The weighted mean of N digests is an exact representation of the group: mean $= \sum_i D_i w_i / \sum_i w_i$. A group of 100 validators can be summarized as one 76-byte value (64-byte weighted mean + 4-byte count + 8-byte variance) with zero information loss. This does not hold for hashes.

Property 3: Set difference identification. When two digests differ, a bloom filter [1] (25 bytes at a 1% per-lookup false positive rate for 20 transactions) lets the aggregator identify which transactions are present in one set but not the other. Bloom filters admit false positives but never false negatives, so a transaction the recipient legitimately needs is never silently dropped; we analyze the false-positive direction in Section 9.5. The aggregator diffs the bloom against the full transaction set and pushes missing transactions in a single message, eliminating the request-response round-trip of traditional state synchronization.

4.3 Threshold Calibration

The distance threshold separating honest-with-partial-observation from Byzantine-with-fabricated-state is calibrated by Monte Carlo simulation. We generate 2000 samples of honest validators missing 1–2 transactions, compute the 99th percentile of the resulting distance distribution, and multiply by a safety margin of 1.2.

4.4 Probabilistic Liveness Bound

Safety depends only on BLS signatures (Section 9.1) and is independent of the distance threshold. Liveness depends on the threshold including at least $2N/3$ honest validators. We derive a bound.

Under the SHA-512 uniformity assumption, each transaction vector $v(tx)$ has coordinates iid Uniform$[0, 1)$ with mean $1/2$ and variance $1/12$. For k missing transactions, the difference $S = v(tx_1) + \cdots + v(tx_k)$ has $\mathbb{E}[\|S\|^2] = 8(k/12 + k^2/4) = 2k/3 + 2k^2$, where the k^2 term is the mean-offset contribution. By Jensen, $\mathbb{E}[\|S\|] \leq \sqrt{\mathbb{E}[\|S\|^2]}$, giving upper bounds $1.63, 3.06, 4.47$ at $k = 1, 2, 3$. Monte Carlo over 50,000 trials gives empirical means $1.61, 3.03, 4.44$, consistent with the bound.

An honest validator with partial observation misses at most $k_{\max} = 2$ transactions; this is the partial-observation model of Sect. 3, and it is what the calibration of Sect. 4.3 samples. The threshold comes from that calibration: the 99th percentile of the sampled honest distance distribution is approximately 4.1, and the 1.2 safety margin gives $\tau = 1.2 \cdot 4.1 \approx 4.9$. Measured over the same calibration sample, the fraction of honest draws whose distance exceeds 4.9 is at most 0.005; this is the exclusion probability conditioned on having an incomplete view. Exclusion requires both an incomplete view, which occurs with probability $p_{\text{miss}} = 0.37$ (Sect. 3), and a conditional exceedance, so the unconditional per-validator exclusion probability is at most $p = 0.37 \cdot 0.005 = 0.00185$.

Let X be the number of honest validators excluded. X is a sum of N independent indicators bounded in $[0, 1]$ with mean $\mu \leq pN$. Liveness fails only if $X \geq N/3$. Since $p \ll 1/3$, applying Hoeffding's inequality with $t = 1/3 - p = 1/3 - 0.00185 \approx 0.33$, so that $2t^2 \approx 0.22$:

$$\Pr\big[X \geq N/3\big] \;\leq\; \exp(-2Nt^2) \;\leq\; \exp(-0.22N).$$

At $N = 100$ this is $\exp(-22) \approx 3 \times 10^{-10}$, below 10^{-9}; at $N = 1000$ it is $\exp(-220) < 10^{-95}$.

5 Application 1: BFT Consensus Protocol

This section applies the primitive to the consensus path itself. The resulting protocol, Proxima, runs two phases: Phase 1 measures agreement geometrically and repairs stragglers, and Phase 2 converts agreement into finality through aggregated BLS commitments; when enough validators already hold the proposed block, the two collapse into a single round trip. Sect. 5.1 presents the flat protocol, Sect. 5.2 routes the same two phases through a tree of small groups, Sect. 5.3 explains why the signature phase can be accelerated but never skipped, and Sect. 5.4 contrasts the resulting group structure with Ethereum's committees.

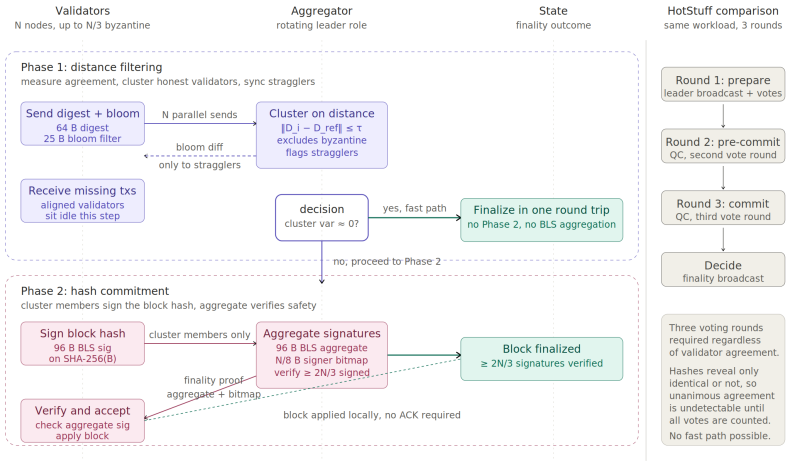

Fig. 2. Two-phase Proxima protocol with speculative fast path. Phase 1 exchanges 64-byte digests and 25-byte bloom filters, plus a conditional 96-byte BLS commitment from each validator whose local state matches the proposal ($64 + 25 + 96$ bytes); the aggregator clusters validators within threshold τ and pushes missing transactions to stragglers via bloom diff. If $\geq 2N/3$ matching commitments are collected in round one, the aggregator aggregates them into the finality certificate (a 96-byte aggregate plus $N/8$-byte signer bitmap, the same object Phase 2 produces) and the block finalizes in one round trip; otherwise Phase 2 collects 96-byte BLS signatures on the SHA-256 block hash from all cluster members. HotStuff (right column) requires three voting rounds regardless of agreement because collision-resistant hashes cannot reveal cluster agreement before votes are counted.

5.1 Flat Protocol

The proposer disseminates the proposed block ahead of Phase 1; validators receive its transactions with some delay relative to the proposer (the partial-observation parameter p_{miss} of Sect. 3), so at digest time each validator holds a local view of the proposal that is usually, but not always, complete.

Phase 1: each validator sends its digest (64 bytes) and bloom filter (25 bytes) to the aggregator (N messages). A validator whose local state matches the proposal attaches to the same message a conditional BLS commitment σ_i on $(\mathit{height}, H(B_{\text{prop}}))$ (96 bytes); validators with incomplete views send digest and bloom only, so the Phase 1 payload is $64+25+96$ bytes from matching validators and $64+25$ bytes from the rest. The aggregator computes the reference digest, measures each validator's Euclidean distance from the reference, and clusters validators within the threshold. The aggregator pushes missing transactions to incomplete validators via bloom diff, then broadcasts cluster assignments.

If the aggregator collects at least $2N/3$ valid commitments on the same hash within round one, it aggregates them and multicasts the finality certificate immediately (fast path). The certificate is byte-identical to the object Phase 2 produces, a 96-byte aggregate signature plus an $N/8$-byte signer bitmap, so single-round finality is covered by Theorem 1 with no modification. If the trigger is not met, the round proceeds exactly as below: clustering, bloom-diff synchronization, and Phase 2 commitment collection from all cluster members. Commitments already received in Phase 1 may be reused in Phase 2; the evaluation conservatively does not model this reuse. The fast path as described applies to the flat protocol; carrying speculative commitments up the tree requires per-group signer bitmaps and is deferred.

Phase 2: cluster members send BLS-signed hash commitments (one 96-byte message each). The aggregator produces an aggregate BLS signature (96 bytes) and a signer bitmap ($N/8$ bytes). The finality proof is multicast to cluster members. Finality requires $2N/3$ matching commitments.

5.2 Tree Protocol

Leaf groups do not need internal BFT. Distance filtering operates at the individual level. A group of 10 with 5 Byzantine excludes the 5 (they are far from the reference) and reports the weighted mean of the 5 honest validators. Groups never fail.

Phase 1 (bottom-up): validators are grouped into leaves of size B (default 10). Each validator sends digest and bloom to its leaf leader (N messages). The leaf leader filters by distance, pushes missing transactions via bloom diff, and sends a 76-byte summary upstream. Internal nodes aggregate child summaries (weighted mean of means is exact). The root checks the global mean against the reference.

Phase 2 (top-down then up): the root broadcasts a commit request down the tree. Each validator that passed the filter sends a BLS commit up. Each internal node aggregates child signatures (BLS is associative). The root produces the final aggregate and broadcasts the finality proof back down.

5.3 Why Phase 2 Is Necessary

The distance filter cannot distinguish an honest validator missing one transaction from a Byzantine validator who replaced one transaction with fraud. Both produce distance ~ 1.6. Phase 2 catches the fraud: the Byzantine validator must sign the correct block hash, which it does not have.

5.4 Comparison with Ethereum Committees

Ethereum 2.0 [12] uses random committees of 128 with per-committee 2/3 BFT attestation. With 30% Byzantine globally, committee failure is the probability of drawing more than $G/3$ Byzantine validators. For G=10: $\Pr[\text{Bin}(10, 0.3) \geq 4] \approx 35\%$. For G=128: $\Pr[\text{Bin}(128, 0.3) \geq 43] \approx 21\%$.

Proxima avoids this failure mode at the per-committee level. There is no per-group vote, so no group fails in the BFT sense. Simulation with 1000 validators, 300 Byzantine (30%), groups of 10: with per-group BFT, 37 of 100 leaves fail and only 499 validators participate; with distance filtering, all 700 honest validators contribute. A leaf of entirely Byzantine validators (rare under random assignment) produces a fabricated summary, but it still has to pass the root's distance check against the reference.

Table 1. Ethereum Committees vs Proxima

	Ethereum	**Proxima**
Group size	128 (BFT)	10 (distance)
Security	Per-group 2/3 vote	Individual distance
Tree depth at N=100K	2–3 levels	5 levels
Groups fail (30% Byz)	$\sim$21%	0%
Summary size/group	128 votes fwd.	76 bytes

6 Application 2: Cross-Shard Consistency

Sharded execution multiplies the state-comparison problem: shards must agree not on a single transaction set but on the overlap between many pairs of sets, and hash-based tools force them to coordinate per transaction. This section shows that digest comparison reduces that coordination to a constant-cost check per shard pair, with resolution work proportional to actual divergence rather than to cross-shard volume.

6.1 The Problem

Cross-shard transactions are the bottleneck in sharded blockchains. Two-phase commit requires 4 cross-shard messages per transaction plus intra-shard BFT

consensus on each coordination message. NEAR's Nightshade [21] reduces this with receipts: the source shard generates a receipt the destination includes in its next block. Receipts achieve lower latency than two-phase commit, but the number of receipts grows linearly with cross-shard transaction volume.

6.2 Digest-Based Verification

Digests enable a different approach. Neighboring shards exchange digests of their overlap-zone transactions once per block. If the distance is zero, both shards processed the same transactions; if it is nonzero, the bloom diff identifies exactly which transactions diverged, and only those require resolution.

The cost is fixed per shard pair: 128 bytes. Conflict resolution scales with actual divergence, not total cross-shard volume.

6.3 Analytical Comparison

Table 2 compares the three methods at 1000 cross-shard transactions per pair, 100 validators per shard, and 95% pre-deadline propagation.

Table 2. Cross-Shard Overhead at 1000 txs, 95% propagation

Method	Messages	Bandwidth	X-shard
2PC	404,000	37,750 KB	4,000
Receipt (NEAR)	101,000	12,625 KB	1,000
Digest (95%)	5,052	987 KB	52

Digest comparison reduces messages by 99% vs 2PC and 95% vs receipts. The 2PC count derives from 4 cross-shard messages per transaction plus 2 intra-shard BFT rounds at the destination, each costing $2N$ messages with N=100: $1000 \cdot (4 + 2 \cdot 2 \cdot 100) = 404{,}000$. At 100% propagation, the digest cost drops to 2 messages (the digest exchange itself); the remaining 5% are identified by bloom diff and resolved individually, which is reasonable on networks with sub-second gossip and multi-second block intervals.

6.4 Multi-Shard Scaling

At 100 shards in a ring topology, with 100 cross-shard transactions per pair and 95% propagation, 2PC uses 4,040,000 messages, receipts use 1,010,000, and digest comparison uses 50,502. Digest overhead scales with conflicts (5% of transactions), not with total volume.

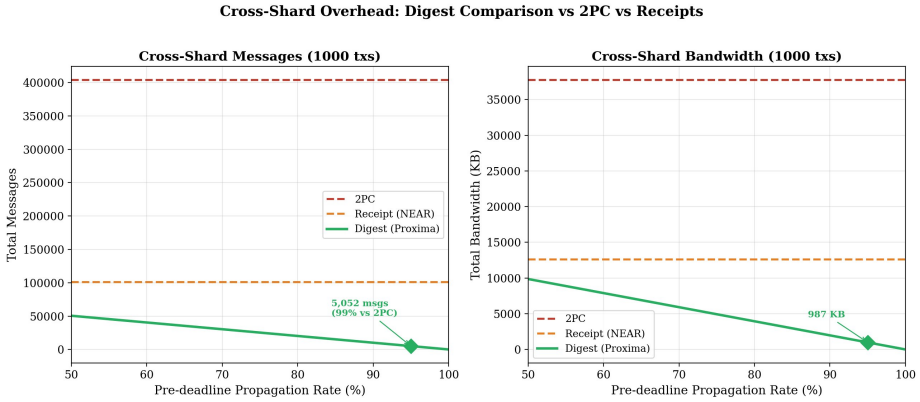

Fig. 3. Cross-shard overhead as a function of pre-deadline propagation rate. 2PC and receipt costs are fixed; digest cost drops sharply as propagation improves. At 95% propagation (marked), digest comparison uses 99% fewer messages than 2PC.

6.5 Why This Requires Digests

Hash-based verification indicates only whether two states are identical, revealing nothing about the magnitude of divergence, so a mismatch forces a fallback to full state exchange or per-transaction coordination. Digest-based verification reports that two states differ by approximately k transactions and, through the bloom diff, identifies which ones, allowing targeted resolution of only the divergent transactions.

7 Application 3: Agreement Quality Measurement

Digests enable two capabilities hash-based protocols cannot provide.

Optimistic fast path. When all validators have complete state, every Phase 1 message carries a matching commitment. The aggregator collects the commitments in round one and issues the finality certificate immediately (Sect. 5.1). HotStuff still runs 3 rounds because it cannot measure agreement quality before votes are counted. The fast-path probability is approximately $(1-p_{\text{miss}})^N$; since this is the probability that every honest validator matches the proposal, while the certificate requires only $2N/3$ matching commitments, it is a lower bound on fast-path engagement. At $p_{\text{miss}} = 0.05$ and $N = 10$ honest validators (the leaf-group regime) it is $\approx 60\%$, rising to $\approx 90\%$ at $p_{\text{miss}} = 0.01$.

Continuous validator reputation. A validator's average distance from the reference over time is a continuous reputation score. Honest validators average 0.3–0.5; Byzantine validators average 7.2. This is richer than binary participation tracking.

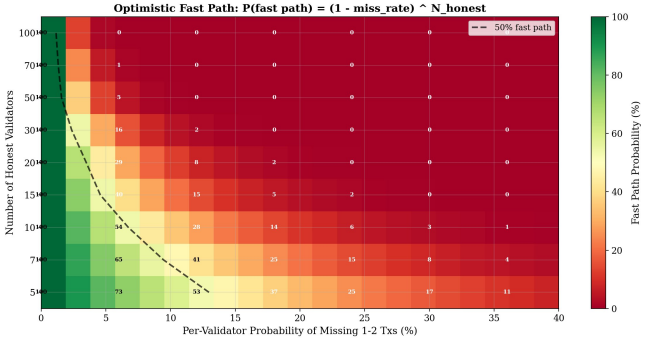

Fig. 4. Lower bound on fast-path probability, $(1 - p_{\text{miss}})^N$, as a function of partial-observation rate and honest validator count; engagement requires only $2N/3$ matching commitments. The fast path dominates on good networks and decays with packet loss.

8 Evaluation

Three questions drive the evaluation: whether two phases and tree routing actually reduce message load at scale, whether distributing BLS aggregation across the tree moves the latency bottleneck, and whether either saving costs Byzantine tolerance. All message counts come from a simulation that increments a counter on every send, using identical byte constants for Proxima, HotStuff, and PBFT; latency is projected from published blst microbenchmarks and a three-tier RTT model rather than measured on a deployed testbed, and the caveats in Sect. 8.3 bound what the projections can claim.

8.1 Message Complexity

All message counts are from simulation with 30% Byzantine and 37% partial observation, tracked by incrementing a counter on each simulated send (Table 3).

Table 3. Messages per block

N	**Prox. Tree**	**Prox. Flat**	**HotStuff**	**PBFT**
1K	2,990	3,348	6,518	2.0M
10K	30,042	33,600	65,180	200M
100K	300,245	335,803	651,800	∼20G

Proxima Tree is 2.2× fewer messages than HotStuff. Message savings come from: two phases instead of three, Byzantine exclusion before Phase 2 (700 commits instead of 1000), bloom sync replacing retransmission round-trips, and tree routing replacing broadcast with compact summaries.

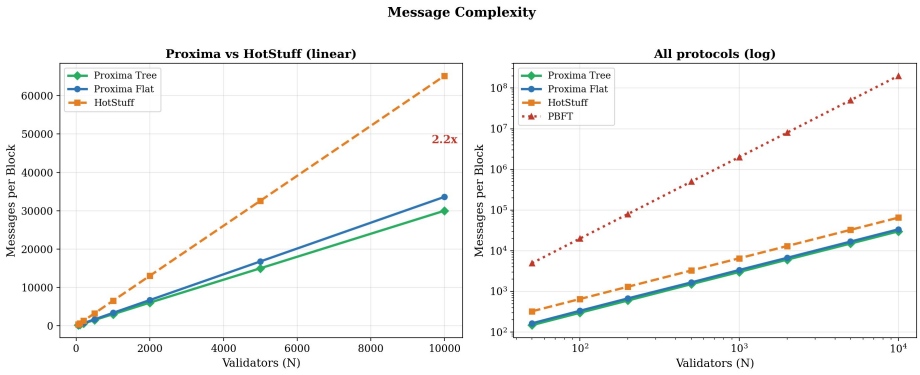

Fig. 5. Message complexity scaling across validator counts. Proxima Tree and Flat both grow linearly but at a lower slope than HotStuff. PBFT is plotted on log scale due to $O(N^2)$ growth.

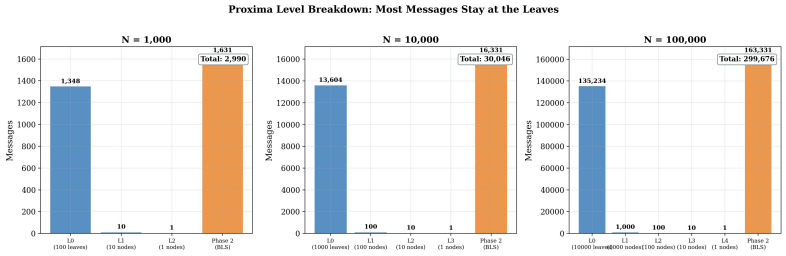

Fig. 6. Message distribution across tree levels at multiple scales. Level 0 (leaves) dominates because every validator sends its vector to a leaf leader. Internal levels (L1+) are nearly invisible because each has branching-factor fewer nodes.

8.2 BLS Aggregation Bottleneck

The tree's primary latency contribution is distributing BLS signature aggregation. We use published blst [23] microbenchmarks (0.05ms per aggregate-add, 1.5ms per aggregate-verify), consistent with production measurements from Ethereum consensus client implementations [20,9] and Ethereum Foundation performance updates [14]. Network RTT uses three tiers: 1ms intra-rack (LOCAL), 80ms intra-region (REGIONAL), 200ms cross-region (GLOBAL), with one tier-appropriate RTT per protocol phase. HotStuff has four phases (PREPARE, PRE-COMMIT, COMMIT, DECIDE): $4 \cdot 200 = 800$ms. Proxima Flat has three round-trip phases (digest exchange, cluster broadcast, commit/finality): $3 \cdot 200 = 600$ms. Proxima Tree pays one LOCAL leaf hop, $(L-2)$ REGIONAL internal hops, and one GLOBAL root hop per phase, doubled to cover both phases: $2 \cdot (1 + 3 \cdot 80 + 200) = 882$ms at L=5 (892ms in the table after small accounting overhead).

Table 4. Projected finality latency at N=100,000. The single-core figures come from the message-counter simulation; the multi-core BLS row is qualitative because parallelizing non-BLS stages depends on the deployment.

	Tree	**Flat**	**HotStuff**
BLS (1 core)	9.9 ms	3,960 ms	17,595 ms
Network RTT (model)	892 ms	600 ms	800 ms
Total finality (1 core)	902 ms	4,561 ms	18,395 ms
BLS only (16 cores)	$\sim$10 ms	$\sim$220 ms	$\sim$940 ms

On a single core the flat aggregator spends 3.96s aggregating 70,000 BLS signatures; HotStuff spends $\sim 3\times$ that across three voting rounds. The tree distributes BLS aggregation across leaves, so each aggregator handles few signatures and the critical-path BLS time is 9.9ms across four levels. With multi-core BLS, flat aggregation drops to $\sim$220ms and each HotStuff round to $\sim$313ms ($\sim$940ms across three rounds); the tree gains nothing from extra cores because each leaf already has only ~ 7 signatures. The tree retains a structural advantage on critical-path BLS time at any core count, but the total finality multi-core picture additionally depends on whether non-BLS stages (e.g., HotStuff's per-validator retransmit handling) parallelize, which we cannot answer without a deployed testbed.

8.3 Caveats

Multi-threaded baselines partially close the gap. The tree's 9.9ms BLS time requires leaf leaders on separate machines; sequential execution on one machine matches flat cost. Conversely, BLS aggregation in flat protocols parallelizes well: a 16-way split reduces flat aggregation to $\sim$220ms and each HotStuff round to $\sim$313ms. The tree gains little from extra cores. The tree retains an advantage

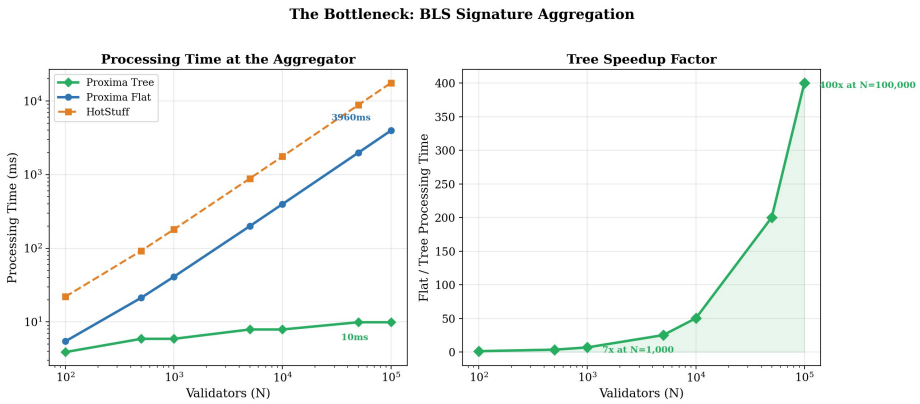

Fig. 7. BLS aggregation bottleneck. Flat and HotStuff processing grows linearly with N; the tree stays roughly constant because each leaf processes at most branching-factor signatures in parallel. Right panel shows the tree/flat speedup factor growing with N.

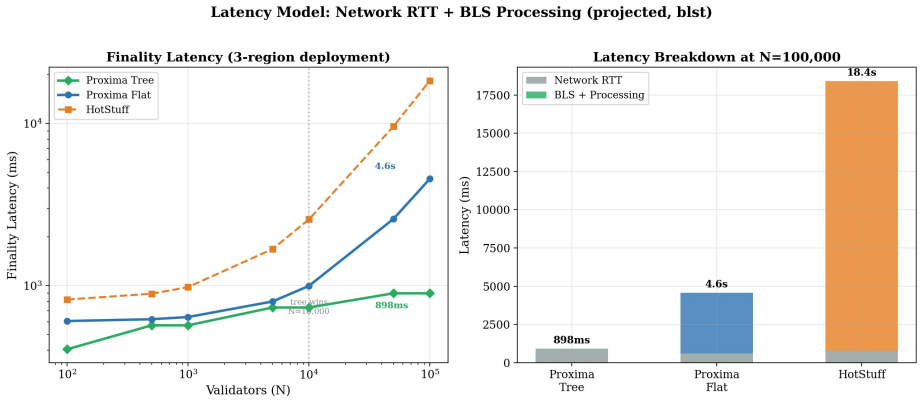

Fig. 8. Total finality latency combining BLS processing and cross-region network RTT. Left: latency vs N. At small N network RTT dominates; at large N BLS processing dominates and the tree's advantage grows. Right: breakdown at N=100,000.

on critical-path BLS time at any core count, but total finality on multi-core hardware additionally depends on whether non-BLS stages parallelize, which we cannot answer without a deployed testbed. The structural advantages, namely message complexity, smaller hierarchical groups, and fast-path finality, are core-count-independent.

Hierarchical HotStuff variants. Kauri [19] applies tree aggregation to HotStuff. A direct comparison would isolate the distance-filtering contribution from tree aggregation. This is left as future work.

Projected, not measured. All latency numbers use published blst constants and standard RTT estimates, not measurements from a deployed system.

8.4 Byzantine Tolerance

The system tolerates up to 33% Byzantine, matching the standard BFT bound. In simulation: 100% consensus success from 0% to 33%, 0% at 35% and above. As Byzantine percentage increases, Proxima gets cheaper (fewer validators in Phase 2) while HotStuff's cost is constant.

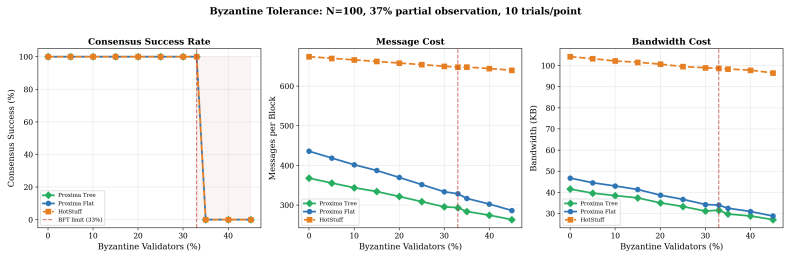

Fig. 9. Byzantine tolerance and cost scaling. All three protocols achieve 100% consensus success from 0–33% Byzantine and fail at 35%+. As Byzantine percentage increases, Proxima's message and bandwidth costs decrease because excluded validators skip Phase 2.

9 Security Analysis

The analysis separates what is proved from what is merely filtered. Safety rests entirely on signed commitments and holds no matter what Phase 1 does; every distance-related mechanism, including the threshold, adversarial digest construction, and bloom false positives, affects only liveness. We prove safety first and then bound each liveness channel in turn.

9.1 Safety

Theorem 1 (Safety). *If fewer than $N/3$ validators are Byzantine, Proxima never finalizes two different blocks at the same height.*

Proof. Finality requires a set C of signed commitments with $|C| \geq 2N/3$, all containing the same block hash h. Assume for contradiction that blocks B and B' both finalize at height k with commitment sets C and C'. Since $|C| \geq 2N/3$ and $|C'| \geq 2N/3$, their intersection $|C \cap C'| \geq N/3$. Every validator in the intersection signed both $H(B)$ and $H(B')$. An honest validator signs at most one hash per height. Therefore the intersection is entirely Byzantine. But $|C \cap C'| \geq N/3$ contradicts $f < N/3$.

Corollary 1. *Phase 1 cannot compromise safety. Incorrect inclusion of a Byzantine validator means it fails to produce a valid BLS signature. Incorrect exclusion of an honest validator reduces commit count (liveness) but no incorrect block is signed.*

The fast-path certificate is itself a set of at least $2N/3$ signed commitments on $H(B)$, collected in Phase 1 rather than Phase 2, so Theorem 1 applies to it verbatim: an aggregator cannot fabricate a fast-path certificate, and a Byzantine aggregator can at most withhold one, delaying the round, which is a liveness effect.

Corollary 2. *Tree routing cannot compromise safety. BLS aggregation is associative. A Byzantine tree node can suppress children's signatures (liveness) but cannot forge signatures.*

9.2 Liveness

Proxima provides liveness under: (1) $f < N/3$ Byzantine; (2) the distance threshold includes at least $2N/3$ honest validators; (3) messages are eventually delivered. Condition (2) is bounded above: the probability that more than $N/3$ honest validators are excluded is at most $\exp(-0.22N)$, negligible at any practical N. Under partial synchrony, a rotating aggregator with timeout ensures progress after GST.

9.3 Collision Resistance

A random Byzantine digest must land within Euclidean threshold τ of the reference in 8D. Treating the relevant support as a cube of side $R \approx 14$ (the half-block distance scale), the probability is the volume ratio of an 8-ball of radius τ to that cube, $\approx 4.06 \cdot (\tau/R)^8$. At $\tau = 4.9$ and $R = 14$, this is $\approx 9.1 \times 10^{-4}$, or about 0.09%. Empirically, Monte Carlo over 10,000 trials shows 0.4% of blocks have a Byzantine validator inside the threshold; the empirical figure being a few times higher than the spherical estimate is the expected sign and magnitude when the digest distribution has heavier-than-uniform tails near the threshold, and serves as a sanity check on the analytical bound. In every case Phase 2 catches it (invalid BLS commitment), so the analysis bounds liveness cost rather than safety.

9.4 Adversarial Transaction Construction

A natural concern is whether a Byzantine validator can craft a transaction set $T' \neq T$ whose digest lands within τ of the honest reference $D(T)$. Distance-preserving digests are deliberately not collision-resistant; a reduction to SHA-512 collision resistance would prove too much. The construction is, at our parameters, computationally easy: each $v(tx)$ is determined by SHA-512(tx) and valid transactions must be signed by accounts the adversary controls (Sybils are free in a permissionless setting, so the binding constraint is gas cost per candidate). The matching condition is roughly $8 \log_2(R/\tau) \approx 12$ bits at $\tau = 4.9$, $R \approx 14$, well within laptop reach for Wagner-style k-list search [24]. *Phase 1 distance filtering therefore contributes no cryptographic security.* The construction is feasible; that is fine, because Phase 2 requires every cluster member to sign a BLS commitment to the SHA-256 block hash $H(B)$, and the Byzantine cannot forge a signature on $H(B)$ where B contains T. Theorem 1 does not invoke any property of $D(\cdot)$. Adversarial construction therefore costs the adversary effort in exchange for no safety violation; the worst-case effect is a failed Phase 2 round, indistinguishable from any other Byzantine signature withholding.

9.5 Bloom Filter False Positives

Bloom filters admit false positives but no false negatives. A false positive causes the aggregator to skip pushing a transaction V needs; V's digest then exceeds τ, V is excluded from the cluster, and V does not sign in Phase 2. The signing set shrinks by one. The chain bloom FP $\rightarrow$ exclusion $\rightarrow$ no signature is wholly a liveness chain: Theorem 1 depends only on collision-resistant hashing and BLS unforgeability, not on cluster membership, so an excluded validator cannot cause a wrong block to clear $2N/3$. The 1% figure is per-lookup, not per-validator: the aggregator queries only flagged stragglers and only against $O(k)$ candidates, where k is the digest-implied gap (typically 1–5), giving a per-validator FP rate of $kp \approx 1$–5%. Filter sizing is linear in transaction count and logarithmic in target FP rate, so scaling the filter with block size to maintain a bounded per-validator rate is a free parameter choice.

10 Implementation

Proxima is implemented in Python as a working BFT blockchain with account-based state, an HTTP node API, an interactive demo, benchmarks, and the evaluation figures, all reproducible from the repository [18]. Digest computation costs one SHA-512 per transaction: the 64-byte output is split into eight 8-byte segments, each reduced modulo 10^4 into a coordinate in $[0, 1)$, and a validator's digest is the vectorized sum of its transaction vectors, traveling as 64 bytes on the wire. BLS signatures run through two interchangeable code paths: py-ecc [13] performs real BLS12-381 operations in the live demo, while large-scale benchmarks substitute hash-based mocks that preserve the 96-byte signature

and 48-byte public-key wire sizes; we cross-validated the paths by running the small-N demo under both and confirming identical per-block message totals. All message counts are tracked by incrementing a counter on each simulated send, using identical byte constants for Proxima, HotStuff, and PBFT, and the latency projections use published blst [23] microbenchmark constants (Sect. 8). On the aggregation side, the implementation computes the reference digest once per block, excludes Byzantine validators before any signature work, synchronizes stragglers with a single bloom-diff push rather than request-response retransmission, aggregates the speculative Phase 1 commitments directly into the fast-path certificate, and in tree mode splits aggregation so that each leaf leader combines only a branching factor's worth of signatures.

11 Limitations and Future Work

Projected latency. All latency numbers use published blst benchmarks and standard RTT estimates. A deployed testbed across 3+ cloud regions would validate the model. The Caveats subsection covers the multi-threaded picture in detail.

No comparison against hierarchical HotStuff. A direct comparison against Kauri would isolate distance filtering from tree aggregation. This is the most important missing evaluation.

Adaptive adversary. The safety proof assumes static Byzantine assignment. An adaptive adversary who corrupts validators after seeing tree assignments requires VRF-based assignment with epoch rotation, which we describe but do not evaluate.

Cross-shard propagation assumption. The 95% propagation rate is consistent with the order-of-magnitude gossip propagation typical at multi-second block intervals but not validated on a sharded testbed.

12 Conclusion

The BFT literature uses collision-resistant hashing for state comparison. We identified three constraints this imposes (mandatory state synchronization, unmeasurable agreement quality, and large hierarchical committees) and showed that distance-preserving digests remove all three. At N=100,000, Proxima Tree uses 2.2× fewer messages than HotStuff (a structural property unaffected by parallelism) and reduces cross-shard overhead by 99% versus 2PC at 95% propagation. Single-core latency is $\sim$ 900ms vs $\sim$ 18s for HotStuff; multi-core BLS narrows the gap considerably but the tree retains an advantage on critical-path BLS time at any core count. Safety is proved. The primitive is general and applies to any BFT protocol that compares state via hashes.

References

1. Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. Communications of the ACM **13**(7), 422–426 (1970)

2. Boneh, D., Lynn, B., Shacham, H.: Short signatures from the Weil pairing. Journal of Cryptology **17**(4), 297–319 (2004)
3. Broder, A.: On the resemblance and containment of documents. In: Proceedings of Compression and Complexity of Sequences (SEQUENCES) (1997)
4. Buchman, E.: Tendermint: Byzantine Fault Tolerance in the Age of Blockchains. Ph.D. thesis, University of Guelph (2016)
5. Castro, M., Liskov, B.: Practical Byzantine fault tolerance. In: Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI) (1999)
6. Charikar, M.: Similarity estimation techniques from rounding algorithms. In: Proceedings of the ACM Symposium on Theory of Computing (STOC) (2002)
7. Danezis, G., Kokoris-Kogias, L., Sonnino, A., Spiegelman, A.: Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus. In: Proceedings of the European Conference on Computer Systems (EuroSys) (2022)
8. Datar, M., Immorlica, N., Indyk, P., Mirrokni, V.: Locality-sensitive hashing scheme based on p-stable distributions. In: Proceedings of the Symposium on Computational Geometry (SCG) (2004)
9. Drake, J.: Pragmatic signature aggregation with BLS. `https://ethresear.ch/t/pragmatic-signature-aggregation-with-bls/2105` (2018)
10. Dwork, C., Lynch, N., Stockmeyer, L.: Consensus in the presence of partial synchrony. Journal of the ACM **35**(2), 288–323 (1988)
11. Ethereum Foundation: Danksharding. `https://ethereum.org/roadmap/danksharding`
12. Ethereum Foundation: Ethereum 2.0 beacon chain specification. `https://github.com/ethereum/consensus-specs`
13. Ethereum Foundation: py-ecc: Python BLS library. `https://github.com/ethereum/py_ecc`
14. Ethereum Foundation: Eth2 quick update no. 8. `https://blog.ethereum.org/2020/05/12/eth2-quick-update-no-8` (2020)
15. Golan-Gueta, G., Abraham, I., Grossman, S., Malkhi, D., Pinkas, B., Reiter, M.K., Seredinschi, D.A., Tamir, O., Tomescu, A.: SBFT: A scalable and decentralized trust infrastructure. In: Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) (2019)
16. Indyk, P., Motwani, R.: Approximate nearest neighbors: Towards removing the curse of dimensionality. In: Proceedings of the ACM Symposium on Theory of Computing (STOC) (1998)
17. Kotla, R., Alvisi, L., Dahlin, M., Clement, A., Wong, E.: Zyzzyva: Speculative Byzantine fault tolerance. In: Proceedings of the ACM Symposium on Operating Systems Principles (SOSP) (2007)
18. Mercier, R.: Distance-preserving digests: A primitive for BFT consensus (extended version). Technical report, `https://github.com/RyanMercier/Proxima` (2026)
19. Neiheiser, R., Matos, M., Rodrigues, L.: Kauri: Scalable BFT consensus with pipelined tree-based dissemination and aggregation. In: Proceedings of the ACM Symposium on Operating Systems Principles (SOSP) (2021)
20. Sigma Prime: Lighthouse update #27: BLS library performance. `https://lighthouse-blog.sigmaprime.io/update-27.html` (2020)
21. Skidanov, A.: Nightshade: Near protocol sharding design. `https://pages.near.org/papers/nightshade/`
22. Spiegelman, A., Giridharan, N., Sonnino, A., Kokoris-Kogias, L.: Bullshark: DAG BFT protocols made practical. In: Proceedings of the ACM Conference on Computer and Communications Security (CCS) (2022)

23. Supranational: blst: BLS12-381 signature library. https://github.com/supranational/blst
24. Wagner, D.: A generalized birthday problem. In: Advances in Cryptology (CRYPTO). pp. 288–304 (2002)
25. Yin, M., Malkhi, D., Reiter, M.K., Golan-Gueta, G., Abraham, I.: HotStuff: BFT consensus with linearity and responsiveness. In: Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC) (2019)